

BLINK OFTALMOLOGIA

Briefing Tecnico Lovable Fase 2

Ponte arquitetural Lovable <-> Agent Lia <-> Kommo/Medware

Versao 1.0 - 27/06/2026

Autor: Fabio Philipe Martins (Diretoria Blink) + Claude Cowork

Destinatario: equipe Lovable (build agent)

1. Sumario Executivo

A Blink Oftalmologia opera um agente conversacional (Lia) que atende pacientes 24h via WhatsApp, integrado a 3 sistemas: Kommo (CRM), Medware (ERP clinico) e Meta Cloud (WhatsApp). Apos cerca de 30 dias de producao, 4 padroes de falha persistem mesmo apos multiplas correcoes:

- Incoerencia na retomada de mensagens enviadas pelo humano - Lia ignora o historico humano-paciente e refaz triagem (caso Larissa/Lis/Samuel).
- Incoerencia apos o agendamento - Lia escreve respostas contraditorias em leads ja em 5-AGENDADO. Caso Thamilla 23811372: afirma consulta confirmada as 11:26 e as 21:33 escreve 'AMIL nao credenciado, encerro?'.
- Falta de envio confiavel de disponibilidade de agenda - Lia diz 'deixa eu reconsultar a agenda' e nunca volta com slots reais. Caso Victor 24147566: 12 promessas vazias em 12 dias.
- Falta de gravacao consistente - Bug C-12 (Kommo retorna success:true sem gravar) e Fix #208 (Lia nao grava Medware autonomamente, depende de intervencao humana ou Cowork).

Este briefing propoe uma arquitetura em 4 fases (~3 semanas) onde o Lovable atua como camada de dados, cache e regras de negocio entre o agente Python (Lia) e os sistemas finais. O agente Lia continua sendo o cerebro conversacional; o Lovable vira o backend de dados que ele consome via 2 endpoints HTTP autenticados.

Resultado esperado pos-implementacao:

- Bug Victor (oferta de slot) - resolvido em 95%.
- Bug Thamilla (memoria/coerencia) - resolvido em 70%.
- Bug C-12 e #208 (gravacao) - resolvidos em 100% por design.
- Bug C-42 (pos-agendamento) - resolvido em 100% por design.

Os 5% e 30% que restam sao limitacoes arquiteturais do agente Python atual (race condition entre turns paralelos - Bug #183) que ainda precisam de pipeline_lock no pipeline.py. Lovable nao resolve essa parte e nem deve tentar.

2. Contexto Operacional Atual

2.1. Arquitetura real (nao a teorica)

Quem responde o paciente HOJE nao e o Salesbot/AI do Kommo. E:

- Agente Python (voice_agent/) rodando em container Docker no Easypanel.
- Webhook do Meta WhatsApp Cloud (8133) ou Evolution API (0710) entrega cada mensagem inbound no agent.
- Agent chama Anthropic Claude (Sonnet 4.5 / Opus 4.6) com prompt mestre em _MASTER_INSTRUCTION.md + tools estruturadas + filtros reativos pos-geracao.
- Agent chama Medware diretamente via medware.py (com cache Redis 5min, retry 1x fail-fast, circuit breaker 3 falhas escalando humano).
- Kommo serve como CRM espelho: humano ve historico, intervem manual, atualiza campos via webhooks bidirecionais.

O Salesbot AI nativo do Kommo esta desativado desde maio/2026.

2.2. Os 4 problemas em detalhe

2.2.1. Memoria entre Lia e humano (Bug Thamilla 23811372)

Em 26/06/2026, a Lia escreveu 2 mensagens diretamente contraditorias num lead ja em 5-AGENDADO:

```
26/06 11:26 - Lia (WhatsApp):
"Sua consulta com a Dra. Karla Delalibera pelo Saude Caixa esta
confirmada para quinta-feira 02/07/2026 as 16:30 na unidade
Aguas Claras." CORRETO

26/06 21:33 - Lia (WhatsApp):
"Thamilla, preciso te corrigir uma informacao: o AMIL nao esta
credenciado na nossa rede... Como prefere seguir?
1) Seguir sem convenio
2) Somente com convenio (encerro o atendimento aqui)"
```

Causa raiz: o campo Kommo N ACEITO CONVENIO=Amil (historico de meses atras) foi lido como sinal do turn atual. O caller_context.py do agent mistura snapshots de momentos diferentes sem timestamping.

2.2.2. Disponibilidade de agenda (Bug Victor 24147566)

De 13/06 a 25/06, em 12 oportunidades, a Lia escreveu variantes de:

```
"Deixa eu consultar a agenda real aqui pra voce - volto em 1 minuto."
"Vou buscar os horarios disponiveis. Me da um minutinho."
"Desculpa a demora, a agenda esta com lentidao no momento."
"Estou trabalhando para conseguir os horarios disponiveis."
"Vou priorizar 16/06 na busca."
... 7 outras variantes
```

E nunca voltou com slots concretos. Em 11 das 12 vezes, atendente humana (Ariany/Stephany) interveio manual.

Causa raiz: a chamada Medware tem latencia variavel (3-15s) e timeout do agent e 12s. Quando estora, agent escapa pra texto livre. Filtros reativos (_viola_oferta_agenda) existem mas estao atras de gate em prod (FILTROS_LEGACY=0).

2.2.3. Pos-agendamento (Bug C-42)

Em status 5-AGENDADO, a Lia nao tem motivo legitimo pra rodar o fluxo de triagem inicial, mas continua entrando nele. Cada turn le fields do Kommo com semantica errada e gera 'opcoes' invalidas como encerrar atendimento de paciente ja com consulta marcada.

Fix imediato (commit 323bbb7, 26/06): adicionados 5-AGENDADO/6-CONFIRMAR/7.CONFIRMADO ao set _STATUS_INATIVOS_IA. Quando lead muda pra essas etapas, ATIVADO IA=Desativado automaticamente. Tampao - confirmacao D-1 vira responsabilidade humana ate filtros C-42 reativos estarem prontos.

2.2.4. Gravacao (Bug C-12 + Fix #208)

Dois sub-problemas independentes:

1. Bug C-12: MCP kommo_update_lead retorna success:true mas custom_fields_values fica vazio. Workaround atual: PATCH direto via Chrome MCP.
2. Fix #208: handle_gravar_agendamento_medware em tools_lia.py era um stub que escrevia flag Redis e delegava pra executor_agendamento.py (arquivo que NUNCA existiu). Por 15 dias seguidos a Lia confirmava agendamento mas nao gravava nada no Medware. Caso Milena 24182212 (20/06): bebe 7 meses, trauma ocular, urgencia - agendamento gravado manual pelo Cowork via mcp__medware__agendar_encaixe.

3. Arquitetura Recomendada

3.1. Visao de 3 hops

Substituir o fluxo atual:

```
ATUAL:
Paciente <-> WhatsApp <-> Agent Python <-> Kommo (fragil)
                                         <-> Medware (lento)
                                         <-> Redis (cache local)
```

Por:

```
PROPOSTO:
Paciente <-> WhatsApp <-> Agent Python <-> Lovable Backend
                                         v (cron)
                                         Supabase
                                         ^ (sync)
                                   Kommo + Medware
```

Pontos importantes da nova arquitetura:

- Agent Python continua intacto como cerebro conversacional. So muda o que ele consome via HTTP.
- Lovable nao recebe webhook direto do WhatsApp - quem recebe e o agent Python. Lovable e backend de dados que o agent consulta.
- Kommo e Medware viram fontes de origem, sincronizadas em batch pelo Lovable. Acesso direto a eles e execucao (so pra escritas que precisam de blindagem).
- Supabase Postgres e a fonte unica de verdade pra estado consolidado de paciente.

3.2. Deciso es arquiteturais nao-obvias

3.2.1. Endpoint publico assinado (nao publico de fato)

Todo endpoint Lovable exposto pra o agent Python tem autenticacao por header:

```
Header obrigatorio:
X-Blink-API-Key: <secret>

Secret reutiliza o KOMMO_WEBHOOK_SECRET que ja esta no
Supabase Secrets (b035819d617d... - 64 chars hex).

Endpoint sem esse header retorna 401 Unauthorized.
```

Sem isso, concorrentes scrapeiam a agenda da Blink em segundos.

3.2.2. Tabela events append-only (nao UPDATE em patients)

Toda mudanca de estado de paciente vira UM evento na tabela events. A tabela patients e apenas snapshot consolidado (view materializada), nunca editada diretamente.

Vantagem: rastreabilidade temporal completa. Quando paciente reclama 'mas eu nao pedi AMIL', ve-se em SQL exatamente quem registrou AMIL, quando e a partir de qual mensagem.

3.2.3. Shadow mode obrigatorio por 48h antes de cada switch

Toda nova entidade (endpoint, tabela, regra) entra primeiro em SHADOW: agent chama o NOVO e o ANTIGO em paralelo, compara resultados, loga divergencias. So faz switch quando divergencia menor que 1% por 48h consecutivas.

Regra introduzida no projeto por incidente do Juiz Haiku 4.5 (Bug 02/06/2026): defesa nova vetando respostas legitimas em producao sem validacao real.

4. Fase 1 - Endpoint Disponibilidade de Agenda

Duracao estimada: 3 dias uteis. Resolve 95% do Bug Victor.

4.1. Objetivo

Substituir a chamada direta do agent Python ao Medware (latencia 3-15s) por chamada ao Lovable (latencia alvo menor que 200ms) que serve da tabela medware_agenda previamente sincronizada.

4.2. Schema do endpoint

4.2.1. Request

```
GET /api/public/agenda/disponiveis
```

Headers:

```
X-Blink-API-Key: <secret>
Accept: application/json
```

Query string:

medico	string	obrigatorio	karla fabricio
unidade	string	obrigatorio	asa_norte aguas_claras
janela_dias	int	opcional	default 14, max 30
hora_inicio	string	opcional	default 07:00 (HH:MM)
hora_fim	string	opcional	default 19:00 (HH:MM)
convenio	string	opcional	ver lista canonica abaixo
lead_id	int	opcional	pra filtrar slots ja ofertados a esse lead (regra E6-B)

4.2.2. Response 200 OK

```
{
  "ok": true,
  "slots": [
    {
      "data_iso": "2026-07-02",
      "hora": "10:30",
      "cod_agenda": 4,
      "cod_medico": 12080,
      "cod_unidade": 5,
      "duracao_min": 30,
      "especialidade": "Oftalmologia Geral"
    }
  ],
  "metadata": {
    "total_disponivel": 8,
    "janela_inicio": "2026-06-27",
    "janela_fim": "2026-06-30",
    "cache_age_seconds": 142,
    "ultimo_sync_medware": "2026-06-27T13:55:00Z"
  }
}
```

4.2.3. Response 503 (Medware indisponível há mais de 10min)

```
{
  "ok": false,
  "erro": "medware_indisponivel",
  "ultimo_sync_ok": "2026-06-27T10:30:00Z",
  "minutos_desde_ultimo_sync": 32,
  "acao_recomendada": "escalar_humano"
}
```

Importante: 503 com ultimo_sync_ok e DIFERENTE de 200 com slots vazios. O agent Lia precisa distinguir esses 2 casos pra escolher a mensagem certa.

4.3. Regras de negócio aplicadas no servidor

3. Karla Delalibera atende: seg/qua/sex em Asa Norte; ter/qui em Aguas Claras.
4. Fabricio Freitas atende: ter/qui em Aguas Claras + Asa Norte conforme escala mensal.
5. Excluir slots entre 12:00 e 13:00 (almoco).
6. Excluir slots com convenio incompativel: paciente Saude Caixa não vê slot reservado a particular se medico restringiu.
7. Excluir slots em blink:slots_ja_ofertados:{lead_id} (Regra E6-B - 10min de reserva temporaria).
8. Excluir slots em dias de ferias do medico (consultar tabela medico_ferias).
9. Default ordenacao: data crescente, hora crescente (slot mais proximo primeiro).

4.4. Tabelas Supabase necessarias

```
-- 1. Espelho da agenda Medware (sincronizado por cron)
CREATE TABLE medware_agenda (
  agenda_id BIGSERIAL PRIMARY KEY,
  cod_agenda INT NOT NULL,
  cod_medico INT NOT NULL,
  cod_unidade INT NOT NULL,
  data DATE NOT NULL,
  hora TIME NOT NULL,
  duracao_min INT DEFAULT 30,
  especialidade TEXT,
  status TEXT DEFAULT 'disponivel',
  cod_paciente_reservado INT,
  medware_sync_ts TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(cod_medico, cod_unidade, data, hora)
);
CREATE INDEX agenda_data_med ON medware_agenda(data, cod_medico);

-- 2. Ferias e escala dos medicos (config manual + cron)
CREATE TABLE medico_ferias (
  id BIGSERIAL PRIMARY KEY,
  cod_medico INT,
  data_inicio DATE,
  data_fim DATE,
  motivo TEXT
);

-- 3. Log de sincronizacoes Medware (saude do cron)
CREATE TABLE medware_sync_log (
```



```

sync_id BIGSERIAL PRIMARY KEY,
started_at TIMESTAMPTZ DEFAULT NOW(),
ended_at TIMESTAMPTZ,
sucesso BOOLEAN,
slots_atualizados INT,
erro TEXT
);

```

4.5. Sincronizacao Medware -> Supabase

- Cron a cada 5 minutos chama Medware Agenda/Listar pra janela de 30 dias.
- Faz upsert na tabela medware_agenda por chave (cod_medico, cod_unidade, data, hora).
- Grava resultado em medware_sync_log (saude monitoravel).
- Se 3 sincronizacoes consecutivas falharem, alerta no Slack #bugs-agent automaticamente.

4.6. Metricas pos-deploy (alvos)

Metrica	Antes	Alvo pos-Fase-1
Latencia media de 'consultar disponibilidade'	8 a 15 segundos	menor que 200 ms
Taxa de 'deixa eu reconsultar' / 1000 turns	Aprox. 6%	menor que 0,5%
Slots oferecidos por conversa de AGENDA	Aprox. 0,3	maior que 1,8
Conversao 'oferta -> confirma slot'	Aprox. 28%	maior que 45%

5. Fase 2 - Memoria Temporal (Tabela Events)

Duracao estimada: 1 semana. Resolve 70% do Bug Thamilla.

5.1. Objetivo

Eliminar o problema 'campo Kommo historico interpretado como sinal atual'. Toda decisao da Lia passa a consultar uma view materializada que separa explicitamente: O QUE E ATUAL x O QUE E HISTORICO.

5.2. Schema de events

```
CREATE TABLE events (
  event_id BIGSERIAL PRIMARY KEY,
  lead_id BIGINT NOT NULL,
  tipo TEXT NOT NULL,          -- ver enum abaixo
  payload JSONB NOT NULL,
  ts TIMESTAMPTZ DEFAULT NOW(),
  source TEXT NOT NULL,        -- lia_inbound|lia_outbound
                                -- |humano|sistema|paciente
  turn_id UUID,                -- mesma rodada de processamento
  snapshot_ctx_known JSONB     -- o que a Lia sabia no momento
);
CREATE INDEX events_lead_ts ON events(lead_id, ts DESC);
CREATE INDEX events_tipo ON events(tipo, ts DESC);

-- Enum de tipos (validacao no application layer):
-- convenio_discutido      paciente mencionou nome de convenio
-- convenio_definido       confirmado e gravado em CONVENIO Kommo
-- slot_ofertado           Lia ou humano enviou slot ao paciente
-- slot_confirmado         paciente disse 'sim' a um slot
-- medware_gravado         agendamento gravado com sucesso
-- kommo_status_change     lead mudou de etapa
-- humano_intervoeio       atendente humano enviou mensagem
-- ia_desativada           ATIVADO IA mudou pra Desativado
-- ia_ativada              ATIVADO IA mudou pra Ativado
```

5.3. View materializada vw_pacient_estado_atual

```
CREATE MATERIALIZED VIEW vw_pacient_estado_atual AS
WITH ultimo_convenio AS (
  SELECT DISTINCT ON (lead_id) lead_id,
    payload->>'valor' AS convenio,
    ts AS convenio_definido_em
  FROM events
  WHERE tipo = 'convenio_definido'
  ORDER BY lead_id, ts DESC
),
ultima_consulta AS (
  SELECT DISTINCT ON (lead_id) lead_id,
    (payload->>'data')::TIMESTAMPTZ AS consulta_data,
    ts AS consulta_confirmada_em
  FROM events
  WHERE tipo = 'slot_confirmado'
  AND (payload->>'data')::TIMESTAMPTZ > NOW()
  ORDER BY lead_id, ts DESC
)
```

```

SELECT
  p.lead_id,
  p.nome,
  p.telefone_e164,
  uc.convenio AS convenio_atual,
  uc.convenio_definido_em,
  ucon.consulta_data,
  ucon.consulta_confirmada_em,
  (ucon.consulta_data IS NOT NULL) AS ja_agendado
FROM patients p
LEFT JOIN ultimo_convenio uc ON uc.lead_id = p.lead_id
LEFT JOIN ultima_consulta ucon ON ucon.lead_id = p.lead_id;

-- Refresh: REFRESH MATERIALIZED VIEW CONCURRENTLY a cada 30s

```

5.4. Integracao com o agent Python

Refactor do caller_context.py:

```

# ANTES (mistura tudo):
def build_caller_context(phone, lead_id):
    kommo_data = kommo.get_lead(lead_id)
    return Context(known=kommo_data['custom_fields'])

# DEPOIS (separa atual de historico):
def build_caller_context(phone, lead_id):
    estado = supabase \
        .table('vw_pacient_estado_atual') \
        .select('*') \
        .eq('lead_id', lead_id) \
        .single().execute()

    historico = supabase \
        .table('events') \
        .select('tipo,payload,ts,source') \
        .eq('lead_id', lead_id) \
        .order('ts', desc=True) \
        .limit(50).execute()

    return Context(
        known=estado.data,          # decisoes usam SO isso
        history=historico.data,     # contexto narrativo apenas
        kommo_raw=kommo.get_lead(lead_id) # fallback
    )

```

5.5. Atualizacao do prompt mestre

Adicionar regra no _MASTER_INSTRUCTION.md:

```

# REGRA TEMPORAL CRITICA

Voce ve dois blocos no contexto:

1. ctx.known - estado ATUAL consolidado do paciente.
   Decisoes SAO baseadas exclusivamente nesse bloco.

```

2. `ctx.history` - eventos passados ordenados por timestamp. Serve apenas pra contexto narrativo. NUNCA tome decisao (de convenio, de horario, de gravacao) baseada em algo do `ctx.history` isoladamente.

Exemplo de uso correto:

```
ctx.known = {convenio_atual: "Saude Caixa", ja_agendado: true}
ctx.history = [{tipo: 'convenio_discutido',
                payload: {valor: 'AMIL'},
                ts: '2026-05-18'}]
```

Resposta: "Sua consulta com Saude Caixa esta confirmada"
(nao menciona AMIL - e historico antigo)

6. Fase 3 - Endpoint de Gravacao Blindada

Duracao estimada: 1 semana. Resolve 100% do Bug C-12 e Fix #208.

6.1. Objetivo

Substituir o orquestracao manual Kommo + Medware no pipeline.py por UM endpoint Lovable que faz as 2 escritas com guardiao Pydantic, GET pos-PATCH e rollback automatico.

6.2. Schema do endpoint

```
POST /api/public/agendamento/salvar

Headers:
  X-Blink-API-Key: <secret>
  Content-Type: application/json

Body:
{
  "lead_id": 24182212,
  "cod_paciente": 6980,
  "cod_agenda": 4,
  "data_iso": "2026-07-02",
  "hora": "10:30",
  "cod_medico": 12080,
  "cod_unidade": 5,
  "cod_plano": 1,
  "cod_procedimento": 303,
  "convenio_validado": false,
  "sinal_pix_comprovado": true,
  "valor_sinal_brl": 335.00,
  "comprovante_pix_url": "https://..."
}
```

6.3. Validacoes servidor (livro Cap. 4.5 - Guardiao)

Regra inegociavel (Bug C-41): rejeita o request se ambos convenio_validado E sinal_pix_comprovado forem false. Sem cobertura financeira, nao grava.

```
# Pydantic strict
class GravarAgendamentoInput(BaseModel):
    lead_id: int = Field(..., ge=1)
    cod_paciente: int = Field(..., ge=1)
    cod_agenda: int = Field(..., ge=1)
    data_iso: str = Field(..., pattern=r'^\d{4}-\d{2}-\d{2}$')
    hora: str = Field(..., pattern=r'^\d{2}:\d{2}$')
    cod_medico: Literal[12080, 12081] # so Karla ou Fabricio
    cod_unidade: Literal[3, 5] # so Asa Norte ou Aguas Claras
    cod_plano: int = Field(..., ge=1)
    cod_procedimento: int = Field(..., ge=1)
    convenio_validado: bool = False
    sinal_pix_comprovado: bool = False

    @field_validator('sinal_pix_comprovado')
    @classmethod
```

```
def exige_cobertura(cls, v, info):
    convenio = info.data.get('convenio_validado', False)
    if not (convenio or v):
        raise ValueError('BUG_C41_RESERVA_SEM_COBERTURA')
    return v
```

6.4. Fluxo de gravacao (transacional)

10. Valida input via Pydantic. Se falhar, retorna 422 + motivo.
11. Inicia transaction Postgres. Insere evento medware_gravacao_iniciada em events.
12. Chama Medware agendar_encaixe (descoberta caso Milena: salvar_agendamento retorna 'horario nao disponivel' falso negativo, encaixe funciona com mesmos params).
13. Se Medware falhar: rollback transaction + insere evento medware_gravacao_falhou + retorna 502.
14. Se Medware OK: PATCH Kommo com 5 campos (1.DIA CONSULTA, 1.UNIDADE, MEDICOS, CONVENIO, STATUS CONVERSA).
15. GET imediato Kommo + comparacao field_id vs valor. Se Kommo 'mentiu' (Bug C-12), retorna 502 + erro 'kommo_silent_failure' + tenta rollback Medware via cancelar_agendamento.
16. Se ambos OK: commit transaction + insere evento medware_gravado + insere evento kommo_atualizado + retorna 200.

6.5. Response 200 OK

```
{
  "ok": true,
  "cod_agendamento_medware": 99887,
  "kommo_updated": true,
  "fields_confirmados": [
    "1.DIA CONSULTA",
    "1.UNIDADE",
    "MEDICOS",
    "CONVENIO",
    "STATUS CONVERSA"
  ],
  "event_id_medware_gravado": 9013,
  "event_id_kommo_atualizado": 9014
}
```

6.6. Por que isso elimina C-12 e #208 por design

Bug #208 (handle_gravar_agendamento_medware era stub): impossivel repetir porque a unica forma de gravar agora e via esse endpoint. Nao da pra 'pular' a chamada - quem decide que existe e o Lovable, nao cada arquivo do agent.

Bug C-12 (Kommo mente em custom_fields): impossivel repetir porque o endpoint sempre faz GET pos-PATCH e levanta erro explicito se valor nao bateu. Nao e tarefa do agent verificar - e tarefa do endpoint blindado.

7. Fase 4 - Vista vw_lead_agendado + Filtros Integrados

Duracao estimada: 3 dias. Resolve 100% do Bug C-42.

7.1. Objetivo

Eliminar definitivamente o cenario Thamilla (Lia em modo triagem em lead ja agendado). View consolidada do Supabase computa ja_agendado baseado em 3 fontes simultaneas. Agent e filtros reativos consultam essa unica fonte.

7.2. View vw_lead_agendado

```
CREATE OR REPLACE VIEW vw_lead_agendado AS
SELECT
  p.lead_id,
  (
    p.status_kommo IN (101507507, 101109455, 106653499)
    OR EXISTS (
      SELECT 1 FROM events e
      WHERE e.lead_id = p.lead_id
            AND e.tipo = 'slot_confirmado'
            AND (e.payload->>'data')::TIMESTAMPTZ > NOW()
    )
    OR (p.consulta_data IS NOT NULL
        AND p.consulta_data > NOW())
  ) AS ja_agendado,
  p.status_kommo,
  p.consulta_data,
  (
    SELECT MAX(e.ts) FROM events e
    WHERE e.lead_id = p.lead_id
          AND e.tipo = 'slot_confirmado'
  ) AS ultimo_slot_confirmado_em
FROM patients p;
```

7.3. Filtros reativos integrados ao Lovable

Antes de cada response Lia sair pra producao, agent chama:

```
GET /api/public/coerencia/validar?lead_id=23811372&texto=<resposta>
```

Response 200 OK (resposta valida):

```
{
  "ok": true,
  "texto_aprovado": "<mesmo texto>",
  "filtros_checados": ["c30a", "c41", "c42", "c36"]
}
```

Response 200 OK (resposta substituida):

```
{
  "ok": false,
  "motivo": "c42_lead_ja_agendado",
  "texto_aprovado": "<texto de fallback canonico>",
  "filtros_disparados": ["c42"]
}
```

Vantagem: regras de coerencia centralizadas no Lovable. Atualizar uma regra nao exige redeploy do agent - basta editar a Edge Function.

7.4. Lista de filtros migrados pra Lovable

Filtro	O que detecta	Origem do bug
c30a	'Deixa eu reconsultar' com ctx.agenda vazio	Victor 24147566
c41	Combinado/Resumo sem cobertura financeira	Milena 24182212
c42	Encerro/triagem em lead ja agendado	Thamilla 23811372
c36	Afirma consulta marcada com 1.DIA CONSULTA passada	Karina 22071351
c35	Dia da semana inventado para a data	Lote de 12 leads

8. Roteiro de Migracao - Shadow Mode

Cada fase entra em producao em 3 estagios obrigatorios.

8.1. Estagio 1 - Shadow puro (24h)

- Agent Python chama Lovable E sistema antigo (Kommo direto / Medware direto) em paralelo.
- Resposta usada: sempre a do sistema antigo. Lovable e so observador.
- Toda divergencia loga no Slack #bugs-agent com payload completo de ambos.
- Critério de saída: divergencia menor que 5% em 24h consecutivas + zero erro 5xx do Lovable.

8.2. Estagio 2 - Switch gradual com fallback (24h)

- Agent chama Lovable primeiro. Se 200 OK em menos de 500ms, usa essa resposta.
- Se Lovable retornar 5xx ou exceder timeout, fallback automatico pro sistema antigo.
- Critério de saída: menor que 1% de fallbacks acionados em 24h.

8.3. Estagio 3 - Switch completo (permanente)

- Sistema antigo e removido da rota critica.
- Mantem funcao de auditoria 1x/dia comparando 100 leads aleatorios.
- Rollback e flag toggle no Easypanel (nao revert de codigo).

8.4. Decisao fast-rollback

Se qualquer estagio mostrar divergencia maior que 15% OU taxa de erro maior que 2%, rollback imediato pra sistema antigo. Investiga em ambiente staging antes de retomar.

9. Riscos e Mitigacao

9.1. Risco - race condition entre turns paralelos

Lovable nao resolve Bug #183 (5 turns sendo processados em paralelo cada um com snapshot ctx diferente).

Mitigacao: pipeline_lock por conversation_key continua sendo responsabilidade do agent Python. Esta em codigo no repo (commit ja em main), falta confirmar deploy em prod e validar via /admin/replay. Sem isso, mesmo com Supabase como fonte unica, 2 turns paralelos podem ler estados ligeiramente diferentes.

9.2. Risco - divergencia cache Medware x Medware real

Cron 5min implica ate 5min de defasagem. Outra clinica/atendente pode pegar slot no Medware enquanto Lovable ainda mostra disponivel.

Mitigacao: validacao on-write - endpoint /api/public/agendamento/salvar faz GET no Medware no momento da gravacao. Se slot foi tomado, retorna 409 Conflict + sugere proximo slot disponivel.

9.3. Risco - endpoint publico de agenda exposto

Mesmo com X-Blink-API-Key, vazamento do secret expoe a agenda da clinica.

Mitigacao: rotacionar secret a cada 90 dias. Adicionar rate limit (100 reqs / min por API key). Log de cada request com IP origem. Em caso de vazamento, rotacao imediata + investigacao de quem teve acesso ao Supabase Secrets.

9.4. Risco - Lovable Edge Function fora do ar

Supabase Edge Functions tem SLA 99.5% (Pro tier). 0.5% = 3.6h/mes de downtime potencial.

Mitigacao: agent Python tem fallback configurado pra cada endpoint. Se Lovable retornar 5xx, agent usa caminho legado (Kommo direto + Medware direto) automaticamente. Operacao degrada mas nao para.

9.5. Risco - custo Supabase escala

Tabela events cresce indefinidamente. Em 6 meses pode ter 500k+ eventos. View materializada lenta.

Mitigacao: particionamento por mes (events_2026_07, events_2026_08). Archive eventos > 12 meses pra storage frio. Refresh da view so nas linhas modificadas (CONCURRENTLY).

10. Glossario - Referencia de Bugs Mencionados

Esta secao lista bugs especificos do projeto Blink referenciados no documento, pra alinhamento de vocabulario com a equipe Lovable.

ID	Descricao	Caso real
C-12	MCP kommo_update_lead retorna success:true mas custom_fields_values fica vazio	Lote diversos leads
C-30A	Lia escreve 'deixa eu reconsultar agenda' com ctx.agenda vazio	Sofia 24158652
C-35	Dia da semana inventado para a data oferecida	Warley + 11 outros
C-36	Lia afirma consulta marcada mas 1.DIA CONSULTA esta no passado	Karina 22071351
C-41	Lia firma reserva sem convenio definido nem sinal Pix	Milena 24182212
C-42	Lia continua em triagem inicial em lead ja agendado	Thamilla 23811372
#183	Race condition entre turns paralelos no pipeline.py	Lote diversos leads
#208	handle_gravar_agendamento_medware era stub	Lote diversos (15 dias sem gravar)

Documentacao completa de cada bug no arquivo CLAUDE.md do repositorio (secao rolling log). Cada bug tem nome do paciente afetado + commit do fix + licao arquitetural.

Proximo passo concreto

Aprovacao deste briefing pela equipe Lovable + criacao de issue por fase no GitHub. Inicio da Fase 1 imediatamente apos aprovacao. Estimativa de entrega completa das 4 fases: 3 semanas corridas (15 dias uteis).

Contato tecnico: Fabio Philipe Martins (oabphi@gmail.com) + Claude Cowork (canal Slack #bugs-agent).